Bonsai CPU Baseline and Bottleneck Scope

Initial benchmark report - Oriol Pont, June 2026

Starting point

As described in the root README, the project began with Bonsai-family 1-bit language models as the acceleration target. Bonsai-1.7B is attractive because it is a general-purpose compact LLM, not a toy benchmark or architectural demo, and its Q1_0 weights expose a specific computation pattern: packed one-bit signs, group scales, and many fixed-weight linear layers. The initial software reference is upstream `llama.cpp`, because it gives a practical CPU-based baseline for edge deployment, and a known-correct execution path for profiling it. The first question after this initial setup was determining where Bonsai actually spends time when running end-to-end inference, from both prefilling (prompt processing) and decoding (autoregressive token generation). To answer that, a benchmark was created to instrument the `llama.cpp`/GGML CPU path and profiled operator time into multiple buckets according to operation type. The benchmark is run across increasing prompt lengths to make the result capture both short and long-context behavior.

Benchmark results

The result files for these tables are, using "pp" prompts processed, and "tg" tokens generated:

- `results/baseline_benchmark/full/prefill-summary.csv`
- `results/baseline_benchmark/full/decode-summary.csv`
- `results/baseline_benchmark/full/pl_1_pp_32768_tg_32.log`, as an example output for a specific run

Prefill summary

Prompt	Tok/s	Q1_0 %	Attn. %	Other %
128	244.45	**88.24**	4.90	6.87
512	407.93	**81.75**	12.70	5.55
2048	343.69	**63.96**	31.74	4.30
4096	269.22	**49.86**	46.93	3.21
8192	155.64	31.33	**63.11**	5.56
16384	108.62	20.12	**78.16**	1.72
32768	56.10	10.70	**88.01**	1.29

Decode summary

Prompt	Tok/s	Q1_0 %	Attn. %	Other %
0	63.28	**93.03**	2.80	4.17
128	63.20	**90.66**	6.49	2.85
512	48.05	**75.61**	15.51	8.87
2048	35.14	**54.94**	42.44	2.61
4096	14.16	37.50	**50.48**	12.02
8192	11.27	34.79	**64.87**	0.34
16384	5.43	12.30	**82.87**	4.83
32768	0.02	3.40	**94.74**	1.86

Percentages are shares of profiled operator time. Q1_0 is `MUL_MAT` with Q1_0 source weights. Attention is `FLASH_ATTN_EXT`, which is the profiled proxy for attention and KV-cache traffic. Decode rows use 128 generated tokens except the 32768-token row, which uses the stored 32-token max-context run.

Bottleneck analysis

The measured split identifies two clear, different regimes. At short contexts, where the model repeatedly applies fixed Q1_0 weights and attention has little history to read. The dependence is primarily arithmetic and packed weight processing: extracting one-bit signs, applying group scales, accumulating dot products, and writing output activations across many layers. On the other hand, at long contexts, attention becomes dominant. This dependence is mainly memory traffic and data movement: the KV cache grows with sequence length, and each generated token must access a larger history.

Since we aim to increase throughput at all different context sizes, the benchmark justifies a dual-target approach. Before assessing the SoC restrictions and architecture, the project should treat these as separate bottlenecks with different causes: one mostly arithmetic over packed weights, the other mostly memory bandwidth and cache traversal.

Weight placement and scalability

An instructive reference is Talos V2, which maps a complete Transformer inference path into RTL and stores fixed weights in ROM-friendly files to achieve extremely high throughput (50k tok/s). This demonstrates the benefit of placing immutable weights beside the datapath and removing their repeated movement. However, Talos deliberately uses a very small character-level microGPT model as a learning tool, not useful for general-purpose LLM inference. FPGA fabric has limited, expensive on-chip memory, while external memory capacity and bandwidth depend on the selected board. Keeping every model weight in FPGA ROM is consequently suitable only for a sufficiently tiny model, or for a system that partitions the model across many accelerator devices. Since the intended approach should remain applicable at different model sizes of the same family of models, the first design should not require the complete model to be resident in the FPGA.

First target: Q1_0 matrix-vector operations

In the profiled `llama.cpp` CPU implementation, a Q1_0 weight block represents 128 weights using 128 sign bits and one FP16 scale. The corresponding activation vector is quantized into Q8_0 blocks. For each dot product, the CPU reads the packed weight bits, expands them into $+1$ or -1 signs, combines them with the signed activation values using SIMD integer dot products, applies the weight and activation scales, and accumulates the result. This is already an optimized CPU kernel, but it still expresses a highly regular fixed-format operation through general-purpose instructions, vector registers, lookup or bit-expansion logic, and repeated loop control.

A specialized path can map the same operation more directly into hardware. **Packed signs can select addition or subtraction without general multipliers**, several activation lanes can be accumulated in parallel, the activation block can be retained while many weight rows pass through, etc. The initial first target is a hardware implementation of the existing Q1_0-by-Q8_0 dot-product semantics, to be compared against the CPU result.

Second target: attention and KV-cache traffic

At long contexts, decode repeatedly traverses an increasingly large KV cache, so attention becomes constrained mainly by the number of bytes that must be read for each new token generated. One possible accelerator contribution is to coordinate the memory hierarchy more carefully, staging and reusing KV data in faster local memories instead of repeatedly accessing slower levels. However, an optimization that only provides more effective bandwidth could eventually be superseded by attaching a higher-bandwidth memory system to the existing inference path. The project therefore focuses on an approach that remains equally useful as hardware bandwidth improves: **reducing the size of the KV cache itself**. A smaller representation lowers the memory capacity required as contexts grow, which in turn reduces the traffic that every level of the memory hierarchy must carry.

The project will therefore focus on KV-cache quantization, initially using ideas from recent SOTA quantization like TurboQuant. TurboQuant is an online vector-quantization method that uses a structured rotation and scalar quantization, with an additional residual correction for inner-product estimation. Applied to the KV cache, this trades extra approximate computation for fewer stored and transferred bits. This trade is potentially suitable for an FPGA: rotation, quantization, packing, reconstruction, and attention-side processing could be pipelined close to the memory stream. This is a second and more experimental target. Compression is only useful if the bytes saved outweigh its compute and metadata overhead, and if the approximation preserves sufficient model quality, which TurboQuant already claims to do.

Architectural scope

The baseline is therefore a RISC-V CPU running Bonsai-family Q1_0 models through a conventional (llama.cpp-based) software inference path. The intended architecture should keep that CPU as the system controller and adds *two* accelerated paths beside it: one for Q1_0 fixed-weight matrix-vector operations, and one for quantization of KV cache for attention mechanisms. The achievable speedup is still unknown and must be measured after implementation, yet the benchmark makes the direction promising given that the proposed accelerators target the operations that majoritarily dominate inference across different context sizes.